



Lab 10: Image flow

MCHA4400

Semester 2 2024

Introduction

In this lab you will use the Lucas-Kanade optic flow algorithm on video recorded from an outdoor scene. You will make use of a robust least squares estimator to find inlier and outlier flow vectors and plot them over the video sequence.

The lab should be completed within 4 hours. The assessment will be done in the lab at the end of your enrolled lab session(s). Once you complete the tasks, call the lab demonstrator to start your assessment.

The lab is worth 5% of your course grade and is graded from 0–5 marks.



GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

1 Camera calibration (1 mark)

Calibrate the DJI Mavic 2 camera using a method similar to your Assignment 1 solution, or a modified Lab 3 solution.

Tasks

- a) Merge the most recent version of following files from previous labs (or Assignment 1) into the Lab 10 working directory:
 - `src/Camera.h`
 - `src/Camera.cpp`
 - `src/rotation.hpp`
- b) Configure the project and build the `lab10` application.
- c) Calibrate the DJI Mavic 2 camera by running the following command, which writes to `data/camera.xml` and displays a box visualisation for each calibration image:

Terminal

```

nerd@basement:~/MCHA4400/lab10/build$ ninja && ./lab10 -c ../data/config.xml
[some failing tests, which we will fix later]
Calibrating camera
Chessboard: boardSize: [10 x 7], squareSize: 0.022
Scanning directory ../data for file pattern "calibration.MOV"
Loading calibration.MOV...done, found 4855 frames
Reading calibration.MOV frame ???...done, detecting chessboard...found
Reading calibration.MOV frame ???...done, detecting chessboard...not found
[etc.]
Reading calibration.MOV frame ???...done, detecting chessboard...found
Reading calibration.MOV frame ???...end of file found
Calibrating camera...done

Calibration data:
    Bit flags: ???
    cameraMatrix: ???
    distCoeffs: ???
    Focal lengths: (fx, fy) = (???, ???)
    Principal point: (cx, cy) = (???, ???)
    Field of view (horizontal): ??? deg
    Field of view (vertical): ??? deg
    Field of view (diagonal): ??? deg
    RMS reprojection error: ???
Display visualisation of calibration results
nerd@basement:~/MCHA4400/lab10/build$

```

- d) Review the calibration results and sanity check them against the [manufacturer specifications](#).

2 Lucas Kanade optical flow (1 mark)

In this task, you will make use of the Lucas Kanade optical flow algorithm. This is implemented in OpenCV as `cv::calcOpticalFlowPyrLK`. We will use the following variable names for each symbol:

$\mathcal{I}[k] \equiv \text{imgk}$	Grayscale image of current frame,
$\mathcal{I}[k-1] \equiv \text{imgkm1}$	Grayscale image of previous frame,
$\mathbb{R}^{2 \times N} \ni \begin{bmatrix} \mathbf{r}_{Q_1/O}^i[k] & \cdots & \mathbf{r}_{Q_N/O}^i[k] \end{bmatrix} \equiv \mathbf{rQOik}$	Feature coordinates in $\mathcal{I}[k]$,
$\mathbb{R}^{2 \times N} \ni \begin{bmatrix} \mathbf{r}_{Q_1/O}^i[k-1] & \cdots & \mathbf{r}_{Q_N/O}^i[k-1] \end{bmatrix} \equiv \mathbf{rQOikm1}$	Feature coordinates in $\mathcal{I}[k-1]$.

Tasks

In `src/image_flow.cpp`, complete the following tasks within the `runImageFlow` function:

- Extract images from the input video file and create a grayscale copy of the i^{th} frame. If the i^{th} frame is to be used, store it in `imgk`.
- Initialise the set of features to track by calling `cv::goodFeaturesToTrack` and store the feature coordinates for k^{th} frame within a variable called `rQOik`. Refine the initial feature set by calling `cv::cornerSubPix` to improve the accuracy of the feature coordinates.
- For every appropriate frame, do the following:
 - Call `cv::calcOpticalFlowPyrLK` to calculate the image flow.

- ii) Filter `rQ0ik` and `rQ0ikm1` by the status output of `cv::calcOpticalFlowPyrLK`, discarding all features that are not found.
 - iii) Display the flow field to a window named LK Demo using `cv::imshow` and `cv::arrowedLine`. Make use of anti-aliasing by passing the appropriate constant from `cv::LineTypes`.
 - iv) Copy `imgk` to `imgkm1` using `cv::Mat::clone` or `cv::Mat::copyTo` and copy `rQ0ik` to `rQ0ikm1`.
 - v) If the number of flow associations falls below some appropriate threshold, re-initialise the set of features to track by calling `cv::goodFeaturesToTrack`.
- d) Run the application on the provided video as follows to generate frames similar to Figure 1, but without outlier colouring:

Terminal

```
nerd@basement:~/MCHA4400/lab10/build$ ninja && ./lab10 ../data/DJI_0121.MOV
[some failing tests, which we will fix later]
Input video: ../data/DJI_0121.MOV
Running image flow

Calibration data:
Display calibration data

maxNumFeatures: ???
minNumFeatures: ???
divisor: ???
imgModulus: ???

Reading frame 0
Initialise/reinitialise feature set.
Found ??? features.
Reading frame 1
...
Calculating flow between frame ??? and 0.
After filtering by status, there are ??? associations.
No. inliers = ???, No. outliers = ???
Reading frame ???
...
Continue to the end of the video
nerd@basement:~/MCHA4400/lab10/build$
```

Observe how the flow field agrees with the motion you would expect to see in the video.

- e) Try resizing the image and skipping frames to see if you can improve the measured flow.

Tip

The input video has a resolution of 2.7k. It may be beneficial to use a lower resolution copy of the input image using `cv::resize`. Use the `int` variable called `divisor`, which will control the level at which the resolution is divided. Set `divisor` to a power of 2 to avoid interpolation artefacts. The intended use of `divisor` is summarised as follows:

- The scaled image is used for all subsequent operations (grayscale conversion, feature identification and tracking, etc.) to improve performance.
- Feature coordinates are scaled by `divisor` when converting between the scaled and original image spaces.

- Since the camera was calibrated at the full image resolution all the **Camera** functions (such as **undistort**) expect image coordinates to be in the original image space, so scaled coordinates are multiplied by **divisor** before using such functions.

**Tip**

The input video is sampled at 60 Hz, therefore, the amplitude of the image flow may be below the noise floor of the flow algorithm in most parts of the provided video (**data/DJI_0121.MOV**). Use the **int** variable called **imgModulus** and make the flow algorithm skip as many frames as dictated by the value of **imgModulus**.

3 Outlier rejection (1 mark)

You will have noticed that the flow measurements have several outliers in some in regions of the image. A common method for rejecting outliers is by looking at the fundamental matrix and solving for motion using a robust least squares estimator like RANSAC. The fundamental matrix can be found using RANSAC to robustly minimise the epipolar error between two frames in OpenCV using `cv::findFundamentalMat`. The **mask** output provides insight into the agreement of each pixel correspondence with the estimated motion between the frames.

The epipolar constraint gives the following relationship between the homogeneous representation of pixels in frames k and $k - 1$.

$$\mathbf{p}[k]^T \mathbf{F}_{k-1}^k \mathbf{p}[k - 1] = 0. \quad (1)$$

The epipolar line in k can be expressed by

$$\mathbf{l}[k] = \mathbf{F}_{k-1}^k \mathbf{p}[k - 1]. \quad (2)$$

The epipolar error, or error perpendicular to the epipolar line, can be calculated from

$$d = \mathbf{n}_p(\mathbf{p}[k])^T \mathbf{n}_l(\mathbf{l}[k]), \quad (3)$$

where we normalise the point and line as follows:

$$\mathbf{n}_p \left(\begin{bmatrix} u \\ v \\ w \end{bmatrix} \right) = \begin{bmatrix} \frac{u}{w} \\ \frac{v}{w} \\ 1 \end{bmatrix}, \quad \mathbf{n}_l \left(\begin{bmatrix} a \\ b \\ c \end{bmatrix} \right) = \frac{1}{\sqrt{a^2 + b^2}} \begin{bmatrix} a \\ b \\ c \end{bmatrix}. \quad (4)$$

We will use the following variable names for each symbol:

$$\begin{aligned} \mathbb{P}^2 \ni \quad \mathbf{l}[k] &\equiv \mathbf{lk} && \text{Homogeneous coordinates of the epipolar line,} \\ \mathbb{P}^2 \ni \quad \mathbf{n}_l(\mathbf{l}[k]) &\equiv \mathbf{nlk} && \text{Normalised homogeneous coordinates of the epipolar line,} \\ \mathbb{R}^{3 \times 3} \ni \quad \mathbf{F}_{k-1}^k &\equiv \mathbf{Fkkm1} && \text{Fundamental matrix.} \end{aligned}$$

Tasks

Within `tests/src/findFundamentalMatrixInlierThreshold.cpp` complete the following:

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/findFundamentalMatrixInlierThreshold.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementations provided.
- Call `cv::findFundamentalMat` with the provided pixel input arrays to generate the fundamental matrix $\mathbf{F}_{k-1}^k$ and the mask vector.
- Create `Eigen::MatrixXd` expressions of the normalised homogeneous points $\mathbf{p}_j[k]$, $\mathbf{p}_j[k-1]$ from the provided OpenCV representations of $\mathbf{r}_{Q_j/O}^i[k]$, $\mathbf{r}_{Q_j/O}^i[k-1]$ for all j . Store each set $\mathbf{p}$ as a $3 \times N$ `Eigen` matrix.
- Form the normalised epipolar lines $\mathbf{n}_l(l[k])$ and store them as columns of a $3 \times N$ `Eigen` matrix.
- Calculate the epipolar error using (3) and store the result in `d`, as a length N `Eigen::RowVectorXd`.
- Run `ninja` and ensure that all tests within `tests/src/findFundamentalMatrixInlierThreshold.cpp` pass.

4 Undistorted optical flow (1 mark)

In this section we will use `cv::undistortPoints` to map between measured points in an image (taken with a camera with lens distortion) and the corresponding points in an ideal image (as if there was no lens distortion). The function `cv::findFundamentalMat` expects such undistorted points as input. Thus, we must undistort the optical flow features before using `cv::findFundamentalMat`.

Tasks

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/undistort.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementations provided.
- In `tests/src/undistort.cpp`, pass appropriate arguments to `cv::undistortPoints`, such that it performs the following mapping:

$$\text{undistortPoints: } \mathbf{r}_{Q/O}^i \mapsto \mathbf{r}_{\bar{Q}/O}^i \quad (5)$$

where $\mathbf{r}_{Q/O}^i$ is the pixel location on the image sensor, and $\mathbf{r}_{\bar{Q}/O}^i$ is the pixel location on the undistorted image sensor.

- In `tests/src/undistort.cpp`, solve $\mathbf{K}\mathbf{r}_{P/C}^c = \mathbf{p}$ for $\mathbf{r}_{P/C}^c$, where $\mathbf{p}$ is the homogeneous representation of $\mathbf{r}_{Q/O}^i$. The solution $\mathbf{r}_{P/C}^c$ may have an arbitrary non-zero magnitude. The $\mathbf{r}_{P/C}^c$ vector is then passed into `vectorToPixel` to compute the image coordinates (with lens distortion) to compare against the original test data.
- Run `ninja` and ensure that all tests within `tests/src/undistort.cpp` pass.

- e) Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/camera.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementations provided.
- f) In `src/Camera.cpp`, complete the implementations of the `Camera::undistort` and `Camera::distort` member functions.
- g) Run `ninja` and ensure that all tests within `tests/src/camera.cpp` pass.

5 Gated optical flow (1 mark)

With the previous tasks completed, we now have the components to do outlier rejection in the optical flow algorithm.

Tasks

Within the `runImageFlow` function in `src/image_flow.cpp`, complete the following:

- a) Use `Camera::undistort` to map $\mathbf{r}_{Q/O}^i[k]$ and $\mathbf{r}_{Q/O}^i[k-1]$ to their respective undistorted locations, $\mathbf{r}_{\tilde{Q}/O}^i[k]$ and $\mathbf{r}_{\tilde{Q}/O}^i[k-1]$.



Tip

Consider how the inputs and outputs of `Camera::undistort` need to be adjusted in order to account for any image scaling defined by the `divisor` variable.

- b) Call `cv::findFundamentalMat` with appropriate input arguments to find the inlier feature correspondences.
- c) Modify the flow vector plot such that the inlier flow vectors are green and the outlier flow vectors are red.
- d) Complete the implementation of the video export functionality¹, to export a video comprised of every frame used in the optical flow calculation with the associated inlier/outlier flow plot. Ensure the output video is exported to `out/<inputname>_<divisor>_<imgModulus>.<inputextension>`. Set the output frame rate such that the output of your video has the same playback duration as the source video.
- e) Ensure the video frames look similar to Figure 1.

¹when the `--export` or `-e` flag is passed from the command line

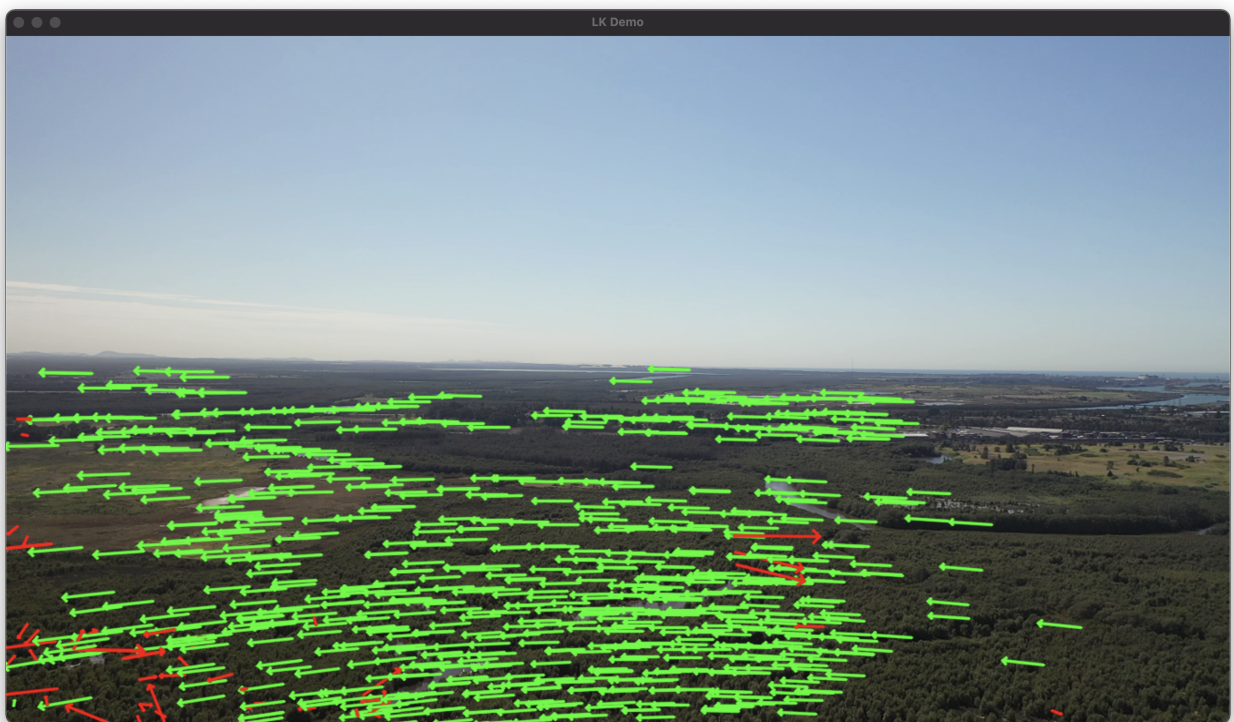


Figure 1: Lucas Kanade image flow with outlier detection.